

AI-Driven Guided Response for Security Operation Centers with Microsoft Copilot for Security

Scott Freitas
scottfreitas@microsoft.com
Microsoft Security Research
Redmond, WA, USA

Amir Gharib
agharib@microsoft.com
Microsoft Security Research
Redmond, WA, USA

Jovan Kalajdjieski
jovank@microsoft.com
Microsoft Security Research
Redmond, WA, USA

Robert McCann
robert.mccann@microsoft.com
Microsoft Security Research
Redmond, WA, USA

Abstract

Security operation centers contend with a constant stream of security incidents, ranging from straightforward to highly complex. To address this, we developed MICROSOFT COPILOT FOR SECURITY GUIDED RESPONSE (CGR), an industry-scale ML architecture that guides security analysts across three key tasks—(1) *investigation*, providing historical context by identifying similar incidents; (2) *triaging* to ascertain the nature of the incident—whether it is a true positive, false positive, or benign positive; and (3) *remediation*, recommending tailored containment actions. CGR is integrated into the Microsoft Defender XDR product and deployed worldwide, generating millions of recommendations across thousands of customers. Our extensive evaluation, incorporating internal evaluation, collaboration with security experts, and customer feedback, demonstrates that CGR delivers high-quality recommendations across all three tasks. We provide a comprehensive overview of the CGR architecture, setting a precedent as the first cybersecurity company to openly discuss these capabilities in such depth. Additionally, we release GUIDE, the largest public collection of real-world security incidents, spanning 13M evidences across 1M incidents annotated with ground-truth triage labels by customer security analysts.

CCS Concepts

• Computing methodologies → Machine learning; • Security and privacy; • Applied computing;

Keywords

Microsoft Copilot, machine learning, cybersecurity, graphs, SOC

ACM Reference Format:

Scott Freitas, Jovan Kalajdjieski, Amir Gharib, and Robert McCann. 2025. AI-Driven Guided Response for Security Operation Centers with Microsoft Copilot for Security. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3701716.3715209>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
WWW Companion '25, Sydney, NSW, Australia
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1331-6/25/04
<https://doi.org/10.1145/3701716.3715209>

1 Introduction

In the rapidly evolving cybersecurity landscape, the sharp rise in threat actors has overwhelmed enterprise security operation centers (SOCs) with an unprecedented volume of incidents to triage [9]. This surge requires solutions that can either partially or fully automate the remediation process. Fully automated systems demand an exceptionally high confidence threshold (e.g., 99%) to ensure correct actions are taken to avoid inadvertently disabling critical enterprise assets. Consequently, attaining such a high level of confidence often renders full automation impractical.

This challenge has catalyzed the development of guided response (GR) systems to support SOC analysts by facilitating informed decision-making. Extended Detection and Response (XDR) products are ideally positioned to deliver precise, context-rich guided response recommendations thanks to their comprehensive visibility across the entire enterprise security landscape. By consolidating telemetry across endpoints, network devices, cloud environments, email systems, and more, XDR systems can harness a wide array of data to provide historical context, generate detailed insights into the nature of threats, and recommend tailored remediation actions.

Guided response challenges. Scalable and accurate GR systems face several key challenges that require a combination of innovative ML system design, and a deep understanding of cybersecurity:

- (1) **Complexity of security incidents.** The extensive variety of security products, each with thousands of custom and built-in detectors, creates a complex incident landscape further compounded by a scarcity of labeled data.
- (2) **High precision and recall.** Analysts require reliable guidance, necessitating systems that deliver high precision and recall across investigation, triaging, and remediation tasks.
- (3) **Scalable architecture.** Generating recommendations at the million-scale across terabytes of data requires a robust and scalable ML architecture.
- (4) **Adaptive to unique SOC preferences.** The system must be able to adapt to specific operational workflows, product configurations, and detection logic of individual SOC.
- (5) **Continuous learning and improvement.** To remain effective against evolving cyber threats and changes in the security product landscape, the system must continuously learn and improve autonomously.

Copilot Guided Response Architecture

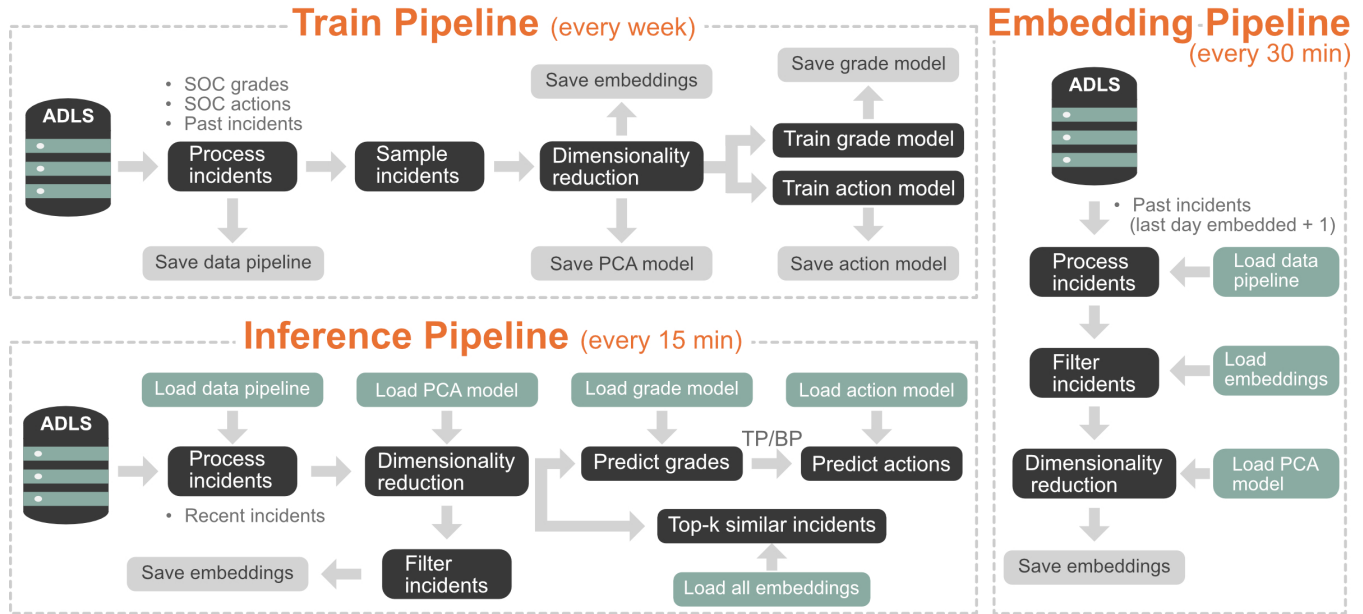


Figure 1: Overview of the COPILOT GUIDED RESPONSE architecture. Train Pipeline: Running weekly, this process trains grade and action recommendation models based on historical SOC telemetry. Inference Pipeline: Running every 15 minutes, this process generates grade, action, and similar incident recommendations for incoming incidents by leveraging the models created in the train pipeline. Embedding Pipeline: Running every 30 minutes until 180 days of historical embeddings exist, this job creates historical embeddings of SOC incidents for the similar incident recommendation algorithm in the inference pipeline.

1.1 Contributions

We introduce COPILOT GUIDED RESPONSE (Fig. 1), an ML framework designed to tackle guided response at scale. Our framework makes significant contributions in the following areas:

- **COPILOT GUIDED RESPONSE (CGR).** The COPILOT GUIDED RESPONSE architecture transforms cybersecurity guided response by detailing the first geo-distributed industry-scale framework capable of processing millions of incidents each day with batch latency of just a few minutes. Our ML system scalably delivers three core SOC capabilities—(1) *investigation*, (2) *triaging*, and (3) *remediation*—seamlessly adapting to a range of scenarios, from single alerts to complex incidents involving hundreds of alerts, where each alert is categorized into one of hundreds of thousands of distinct classes, with new classes continuously added.
- **Largest Cybersecurity Incident Dataset.** We introduce GUIDE, the largest publicly available collection of real-world cybersecurity incidents under the permissive CDLA-2.0 license. This extensive dataset includes over 13 million pieces of evidence across 1.6 million alerts and 1 million incidents annotated with ground-truth triage labels by customer security analysts, making it an unparalleled resource for the development and evaluation of GR systems and beyond. By enabling researchers to study real-world data, GUIDE advances the state of cybersecurity and supports the development of next-generation ML systems.

- **Extensive Evaluation of CGR.** We provide a comprehensive evaluation of CGR's performance, focusing on its industrial applicability rather than comparisons with alternative designs or competing security products, which are often undisclosed. The release of the GUIDE dataset enables researchers to explore new architectures for maximum performance. Our evaluation spans internal testing, expert collaborations, and customer feedback. Internal assessments on hundreds of thousands of unseen incidents show triage models achieve 87% precision and 41% recall, while action models reach 99% precision and 62% recall. Collaboration with Microsoft security experts confirms the efficacy of our similar incident recommendations, with 94% of incidents deemed relevant, and 98% of incidents containing one or more recommendations. Customer feedback further underscores its effectiveness, with 89% of interactions rated positively.
- **Impact to Microsoft Customers and Beyond.** CGR is integrated into the Microsoft Defender XDR product, a leader in the market [30], and is deployed to hundreds of thousands of organizations worldwide. The introduction of CGR to Microsoft Defender XDR has significantly enhanced the operational capabilities of SOC's by streamlining the decision-making process and providing actionable insights across investigation, triaging, and remediation tasks. As a result, Microsoft Defender XDR customers benefit from a more resilient security response, fortified by adaptive ML-driven guided responses that are tailored to the nuances of their specific security environments.

Term	Definition
Alert	Potential security threat that was detected
Detector	A security rule or ML model that generates alerts
Entity	File, IP, etc. evidence associated with an alert
Correlation	A link between two alerts based on a shared entity
Incident	Related alerts that are correlated together
SOC	Security operation centers (SOC) protect enterprise organizations from threat actors
XDR	Extended Detection and Response (XDR) platforms are used by SOC to protect organizations across the entire enterprise landscape

Table 1: Terminology and definitions.

2 Background

We provide an overview of MICROSOFT COPILOT FOR SECURITY and review literature relevant to guided response. To enhance readability, Table 1 details the terminology used in this paper.

2.1 Microsoft Copilot for Security

Microsoft Copilot for Security is an AI-driven solution that enhances security professionals' workflows by offering real-time insights and recommendations across Microsoft Defender XDR, Microsoft Sentinel, and Microsoft Intune. At launch, it introduced five skills: incident summarization, script analyzer, incident report, Kusto query assistant, and guided response [8]. While the first four leverage LLMs with security-specific plugins and post-processing [31], guided response includes three machine learning sub-skills—grade recommendation, action recommendation, and similar incident recommendation—tailored to SOC preferences for precise, context-specific insights.

2.2 Guided Response

With the market introduction of MICROSOFT COPILOT FOR SECURITY GUIDED RESPONSE, the concept of guided response was formally defined as “*machine learning capabilities to contextualize an incident and learn from previous investigations to generate appropriate response actions*” [17]. Our analysis of academic and industry literature contextualizes relevant contributions within the domain of guided response into three distinct categories: (1) investigation that suggests next steps for further analysis; (2) triaging to determine whether an incident is a true positive, false positive, or benign positive (e.g., informational); and (3) remediation which proposes specific response actions to contain and resolve incidents.

Investigation. Assisting security analysts in their investigation of incidents is a pivotal aspect of cybersecurity. Machine learning assisted investigation typically encompasses: (1) similar incident identification [23, 28, 46], (2) investigation assistance [14, 35, 38], and (3) playbook recommendation [5, 25, 26]. Our research focuses on similar incident recommendation, an understudied topic in the context of SOC. While a few industry solutions offer similar incident identification capabilities through methods such as comparing and counting identical artifacts [32, 33], details regarding their

methodologies and performance metrics are scarce. We address this gap by detailing the first industry-scale architecture for similar incident recommendation in SOC.

Triaging. Incident triaging is a vital and time intensive task typically performed by junior analysts to identify incidents requiring further investigation. This involves prioritizing incidents for deeper review [1, 2, 4, 29, 37] and filtering them based on the likelihood of being true versus false positives [6, 18, 19, 42]. While some industry solutions use ML to automate triaging [21], there is limited public information on their architecture or performance. Although some companies provide insights [12, 36, 42], these are often limited to controlled scenarios and lack critical deployment details. Our work addresses these gaps by detailing a scalable, adaptable ML triaging architecture that is deployed globally to thousands of SOC.

Remediation. The majority of remediation research centers on intrusion response systems (IRS), designed to notify SOC analysts or dynamically respond to detected intrusions. These systems employ various decision-making models, including rule-based approaches [13], multi-objective optimization [27, 41, 45], game theory [20, 39, 47], human-in-the-loop [34], reinforcement learning [3, 24, 43], and many others [7, 22, 40] to determine optimal responses based on the system state, nature of the attack, and countermeasure impact. Despite advancements, there is limited discussion on how to scale these systems to handle industry demands, such as managing millions of incidents, handling complex scenarios with numerous alerts, and customizing responses for SOC preferences. Our research bridges this gap, enhancing the scalability and transparency of industry-scale guided remediation systems.

3 Architecture Overview

We detail the COPILOT GUIDED RESPONSE (CGR) architecture, organized around three key pipelines: train, inference, and embedding, as illustrated in Figure 1. We utilize PySpark's distributed computational engine whenever possible, while reserving Python for last mile recommendation tasks that do not have PySpark support. Below, we detail how these pipelines synergize to guide security analysts through the processes of investigating, triaging, and remediating security incidents across the enterprise landscape.

- **Train pipeline (Section 4).** Running weekly, this process trains the grade and action recommendation models using historical SOC telemetry to provide tailored responses. This procedure is detailed across ten steps, T1 through T10.
- **Inference pipeline (Section 5).** Operating every 15 minutes, this pipeline generates grade and action recommendations for incoming incidents by leveraging the models developed in the train pipeline. Additionally, it provides similar incident recommendations by matching new incidents with historically similar incidents generated in the embedding pipeline.
- **Embedding pipeline (Section 6).** This pipeline runs every 30 minutes until 180 days of incident embeddings are generated. These embeddings form the foundational data that allows the similar incident recommendation algorithm in the inference pipeline to effectively identify similar incidents.

To adhere to privacy regulations, CGR is replicated across geographic regions, utilizing Synapse to ensure consistency and compliance. Consequently, the following sections focus on development from the perspective of a single geographic region.

4 Train Pipeline

COPILLOT GUIDED RESPONSE's training architecture is detailed across two subsections—Section 4.1 which presents the key steps to collecting and preparing the data; and Section 4.2 which discusses the model training and validation process. A step-by-step overview of the entire training process is provided in Algorithm 1.

4.1 Preprocessing

We detail the ten step process (T1-T10) for creating alert and incident dataframes leveraged across all three pipelines.

T1—Feature engineering. We collect alert telemetry from multiple Azure Data Lake Storage (ADLS) tables and join them into a PySpark alert dataframe. Each row in the alert dataframe contains columns for unique alert and incident identifiers, complemented by customer-provided grade and remediation action, when available. Additionally, each row contains 5 categorical feature columns—OrganizationId, DetectorId, ProductId, Category, and Severity—along with 67 engineered numerical feature columns, developed in close collaboration with Microsoft security research experts. We retain rows that lack a customer grade or action, as these alerts can merge with other alerts to form incidents that do contain labels.

T2—Feature space compression. Before converting the categorical columns to one-hot-encoded representations, we must address the challenge of high cardinality in the DetectorId and OrgId columns. In various geographic regions, DetectorIds can exceed 100k and OrgIds can reach up to 50k, creating an extremely large and sparse feature space that often leads to failures during dimensionality reduction in the PySpark cluster. To mitigate this, we aggregate the feature space by substituting infrequent values—those associated with fewer than 10 alerts—with a generic value. This method, while resulting in some information loss, ensures the system remains within the computational boundaries of the cluster.

T3—One-hot-encoding. With the preliminary adjustments to our feature space, we can convert all 6 categorical feature columns into their one-hot-encoded (OHE) form. This transformation includes key columns such as OrgId, ProductId, and DetectorId, which allows the models to capture SOC-specific tendencies as well as product and detector specific characteristics that evolve over time. We bifurcate the data and establish a secondary PySpark alert dataframe that only contains alerts with remediation actions, while retaining all alerts in the original dataframe. Finally, we store the PySpark OHE pipeline in an Azure Blob Storage container so that it can be used in the inference process to transform the categorical columns.

T4—Forming incidents. To enhance our ability to make precise incident-level triaging decisions and investigation recommendations, we create a separate incident dataframe. This is achieved by aggregating alert rows based on shared IncidentIds from the alert dataframe containing all alerts, and summing their respective

Algorithm 1: COPILLOT GUIDED RESPONSE Training

Input: Alert data A , minimum cardinality c , principal components k , max incidents sampled per IncidentHash m , max incidents stored per IncidentHash s , grid search parameters $\mathcal{G}$

Output: Incident embeddings I & trained models $\mathcal{M}_t, \mathcal{M}_r$

Feature Engineering

```

 $A \leftarrow \text{FeatureEngineering}(A)$  ; // T1
 $A \leftarrow \text{FeatureSpaceCompression}(A, c)$  ; // T2
 $A \leftarrow \text{OneHotEncoding}(A)$  ; // T3
 $I \leftarrow \text{FormIncidents}(A)$  ; // T4
 $I \leftarrow \text{SampleIncidents}(I, m)$  ; // T5
 $I, A \leftarrow \text{PCA}(I, k), \text{PCA}(A, k)$  ; // T6
StoreEmbeddings( $I, s$ ) ; // T7

```

Model Training

```

 $I', A' \leftarrow \text{ConvertToPandas}(I, A)$  ; // T8
 $\mathcal{M}_t \leftarrow \text{TrainTriageModel}(I', \mathcal{G})$  ; // T9
 $\mathcal{M}_r \leftarrow \text{TrainRemediationModel}(A', \mathcal{G})$  ; // T9
ValidateAndStoreModels( $\mathcal{M}_t, \mathcal{M}_r$ ) ; // T10

```

numerical columns. For incidents with multiple grades, the majority label is applied, with ties going to the true positive class. We remove any incidents without a triage grade at this stage.

T5—Sampling incidents. Given that incident processing steps are significantly more memory-intensive than remediation actions in the alert dataframe, we employ random sampling on the incident dataframe to mitigate out-of-memory issues during downstream processing steps. This sampling strategy involves creating a unique IncidentHash identifier for each incident by arranging the DetectorIds of an incident into an ordered list and hashing it using SHA1. We can then cap the number of incidents for each unique IncidentHash and triage grade to a maximum of 1,000.

T6—Dimensionality reduction. We independently apply principal component analysis (PCA) to both the incident and alert dataframes, each containing tens of thousands of columns. PCA is chosen for its performance and availability within PySpark's native machine learning library, which supports distributed computing and allows for efficient feature space reduction. This mitigates the risk of out-of-memory errors that occur when centralizing large dataframes on the primary PySpark node for subsequent scikit-learn model training. Our objective is to condense the feature space to k principal components that captures 95% of the original variance in each dataframe. Empirically, we find that setting $k = 40$ meets this requirement. The resulting PCA weights are saved in an Azure Blob Storage container, enabling reuse within the inference and embedding pipelines.

T7—Store embeddings. The final step is to save the incident embeddings to an ADLS table to enhance similar incident recommendations within the inference pipeline. A strategic product decision dictates that only the top five most similar incidents are displayed for any given incident. Consequently, we only store up to five instances of each incident, categorized by triage grade and the unique set of DetectorIds that comprise the IncidentHash.

4.2 Model Training

We train two models—a triage model to predict incident grades and an action model to determine remediation actions for incident-related alerts—across three key steps: (1) converting the PySpark dataframes to Pandas and performing a stratified train-val-test split; (2) optimizing random forest models with a grid search; and (3) validating new models against previous versions before storage. While PySpark’s native MLlib is an option, our experiments show it results in a 10% decrease in Macro-F1 score compared to scikit-learn due to missing core capabilities. For instance, MLlib’s random forest model is limited to a depth of 30, significantly constraining its ability to capture complex patterns.

T8—Dataset formation. We begin by converting the alert and incident PySpark dataframes created in Section 4.1 into Pandas. For each Pandas dataframe, we conduct a standard 70-10-20 train, validation, and test set split of the data, stratified by grade and action labels, respectively.

T9—Training process. We select a random forest model due to its efficiency on our CPU-based PySpark infrastructure and its reliable performance on tabular data. We conduct a grid search over four key model parameters: $n_estimators = \{100, 200, 300, 400\}$, $max_depth = \{30, 50, 75, 100\}$, $min_samples_split = \{5, 10, 15\}$, and $class_weight = \{'balanced', None\}$, and select the model with the highest macro-F1 score on the validation set.

T10—Validation and model storage. Once the best model has been identified for both triage and remediation tasks, we compare them with previous models to ensure quality between training cycles remains within a few percentage points. We do not require each model to have a higher macro-F1 score since new detectors and security products are onboarded over time, which can cause fluctuations in performance. After validation, the models are saved to an Azure Blob Storage container for use in the inference pipeline.

5 Inference Pipeline

Building on the infrastructure in Section 4, the inference pipeline processes batched data to provide guided response recommendations (Sec 5.1). For each new or updated incident, the trained models and historical embeddings operate across three phases: triage to predict grades (Sec 5.2), investigation to find similar incidents (Sec 5.3), and remediation to recommend response actions (Sec 5.4). Recommendations dynamically update as incidents evolve and are stored in a table, ensuring rapid access for customers. Algorithm 1 provides a step-by-step overview.

5.1 Preprocessing

The initial phase of the inference pipeline prepares real-time batched alert data, retrieving the last 15 minutes of telemetry and loading it into a PySpark dataframe. These alerts are then processed using the feature space compression and one-hot encoding techniques outlined in Section 4.1. Next, we bifurcate the alert data into two distinct PySpark dataframes—one dedicated to generating remediation predictions and the other for aggregating alerts into incidents for similar incident and triage recommendations, utilizing the aggregation process described in Section 4.1. Afterwards, we apply the

Algorithm 2: COPILOT GUIDED RESPONSE Inference

Input: Batched alert data A , max incidents stored per IncidentHash s , triage and remediation models

M_t, M_r with confidence thresholds c_t, c_r

Output: Triage, investigation, and remediation recs

Preprocessing

```

 $A \leftarrow \text{FeatureEngineering}(A)$  ; // T1
 $A \leftarrow \text{FeatureSpaceCompression}(A)$  ; // T2
 $A \leftarrow \text{OneHotEncoding}(A)$  ; // T3
 $I \leftarrow \text{FormIncidents}(A)$  ; // T4
 $I, A \leftarrow \text{PCA}(I, \text{PCA}(A))$  ; // T6
 $\text{StoreEmbeddings}(I, s)$  ; // T7
 $I', A' \leftarrow \text{ConvertToPandas}(I, A)$ 

```

Triage Recommendations

```

 $\mathcal{R}_{tri} \leftarrow M_t(I)$ 
 $\mathcal{R}_{tri} \leftarrow \text{FilterByConfidence}(\mathcal{R}_{tri}, c_t)$ 

```

Investigation Recommendations

```

 $H \leftarrow \text{RetrieveHistoricalEmbeddings}(180 \text{ days})$ 
 $\mathcal{R}_{inv} \leftarrow \emptyset$ 
foreach  $I_{new} \in I_{tri}$  do
     $\mathcal{R}_{inv} \leftarrow \mathcal{R}_{inv} \cup \text{ExactHashMatch}(I_{new}, H)$ 
     $\mathcal{R}_{inv} \leftarrow \mathcal{R}_{inv} \cup \text{CosineSimilarityMatch}(I_{new}, H)$ 
     $\mathcal{R}_{inv} \leftarrow \text{SelectTopK}(\mathcal{R}_{inv}, 5)$ 

```

Remediation Recommendations

```

 $\mathcal{R}_{rem} \leftarrow M_r(A)$ 
 $\mathcal{R}_{rem} \leftarrow \text{FilterByConfidence}(\mathcal{R}_{rem}, c_r)$ 
 $\mathcal{R}_{rem} \leftarrow \text{IdentifyEntities}(\mathcal{R}_{rem})$ 
 $\mathcal{R}_{rem} \leftarrow \text{AggregateRecommendations}(\mathcal{R}_{rem}, I)$ 

```

latest PCA models from the training pipeline to reduce the dimensionality of both dataframes to form alert and incident embeddings. The incident embeddings are stored in an ADLS table to enhance the similar incident recommendations, with a limit of five instances per incident, categorized by triage grade and DetectorId. Finally, we convert both PySpark dataframes into Pandas dataframes to facilitate subsequent triage, investigation, and remediation processes.

5.2 Triage Recommendations

We leverage the incident embeddings produced during preprocessing, along with the latest version of the triage model, to generate triage recommendations. Each incident is evaluated by the model and given a prediction of true positive (TP), false positive (FP), or benign positive (BP), the latter considered as an informational incident. The confidence of each recommendation is assessed against a precision threshold of 0.9 to ensure that only reliable recommendations are sent to SOC analysts.

5.3 Investigation Recommendations

We utilize the incident embeddings produced during the preprocessing step to generate recommendations for similar incidents. This process begins with the retrieval of historical incident embeddings from our Azure Data Lake Storage (ADLS), going back up to 180 days. These embeddings capture past incidents in a vectorized format, enabling efficient comparison. The core of our approach is

to match new incidents with historically relevant incidents within the same organization through a three-step matching process:

- (1) **Exact hash matching.** We begin by identifying historical incidents that share the same IncidentHash and triage recommendation. If less than five matches are found, we take incidents with the same IncidentHash but differing triage recommendations.
- (2) **Approximate matching with cosine similarity.** If less than five exact matches were found, we search for historical incidents based on the cosine similarity of their embeddings. This approach helps to identify incidents that share significant characteristics with the current incident.
- (3) **Top-k similar incident selection.** We select the top-k most similar incidents, up to a maximum of five. Exact and cosine similarity matches are ordered, with a higher priority given to exact matches to ensure the most germane comparisons, and ties going to the most recent incident.

5.4 Remediation Recommendations

Using alert embeddings from preprocessing and the latest remediation model, we generate targeted response actions—contain user, isolate machine, or stop virtual machine—for each alert with confidence above a 0.9 precision threshold. The system identifies entities (e.g., users, devices, VMs) using encoded rules based on security domain knowledge, and then aggregates the individual alert recommendations into comprehensive incident recommendations.

6 Embedding Pipeline

We generate historical incident embeddings that allow the similar incident recommendation algorithm to leverage up to 180 days of historical data when making recommendations. Due to the limitations of the training pipeline in processing huge volumes of incident telemetry across regions, we developed a specialized mechanism to generate historical embeddings. This ensures that our similar incident recommendation algorithm rapidly reaches comprehensive historical coverage each time the inference pipeline is executed.

Continuous embedding generation. The embedding pipeline operates in a continuous loop, with each iteration processing data one day further back than the last. Leveraging the preprocessing steps outlined in Section 5.1, we integrate a deduplication process that loads historical incident embeddings and compares incident hashes and triage recommendations to eliminate redundancy. We store any IncidentHash and triage recommendation pairs from the current batch, including those without a triage grade, provided they do not exceed five stored embeddings—aligning with our policy of recommending no more than five similar incidents at a time. New incident embeddings are then saved to the ADLS table for use in the inference pipeline. This procedure repeats until we have 180 days of historical incident embedding telemetry.

7 Experiments

We evaluate CGR’s performance across three tasks: (1) triaging, assessing the model’s ability to classify incidents as benign, malicious, or informational; (2) investigation, analyzing the relevance of similar incident recommendations; and (3) remediation, evaluating its ability to predict effective threat mitigation actions.

7.1 Setup

We present results for the triage and remediation models across a sample of 12 regions, where each regional dataset is divided into three stratified subsets: training (70%), validation (10%), and testing (20%). Each run of model training job includes two key parameter optimizations, performed independently for incident triage and alert remediation embeddings:

- (1) **PCA component selection.** We retain the top-k principal components capturing 95% of the data variance, with k ranging up to 100. While the optimal k varies across runs and regions, we find that $k = 40$ performs well across most scenarios.
- (2) **Random forest parameter tuning.** We optimize key random forest parameters using a grid search (see Section 4.2). We select the model with the highest macro-F1 score on the validation set and report precision and recall metrics, as is standard for imbalanced datasets [10, 11, 15, 16].

Table 2 summarizes model training and performance statistics over two weeks across select regions, including the count of unique DetectorIds, and alert/incident volumes. Regional variations are significant, with up to 31k detectors across thousands of organizations, complicating the training process. Incident size distribution (Figure 2) is long-tailed, with most incidents comprised of a few alerts, with larger incidents representing a greater challenge for triage and similar incident recommendations due to their rarity.

Limitations. This work focuses on presenting a scalable, unified framework for incident triaging, remediation, and similar incident recommendations, rather than evaluating alternative models or competing security products, which are often undisclosed. The release of the GUIDE dataset enables researchers to explore and optimize new architectures for maximum performance. In addition, CGR is limited to providing recommendations for existing detectors and does not address zero-day or other unmonitored attack vectors.

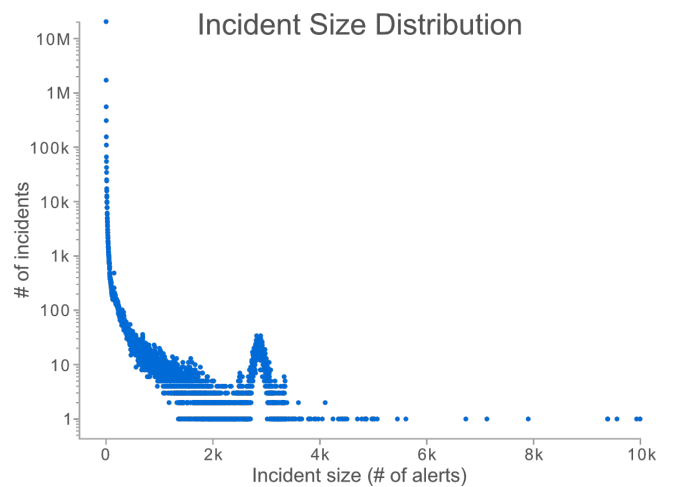


Figure 2: Sampled distribution of incident size—measured by the number of alerts per incident—exhibits a long-tailed pattern where the majority of incidents have only a few alerts.

Train Statistics				Triage Results							Remediation Results						
Region	# Rules	# Alerts	# Inc	Supp	% TP	% FP	% BP	Pr	Re	F1	Supp	% CA	% ID	% VM	Pr	Re	F1
1	31k	18.4M	5.1M	49k	16	32	52	.86	.85	.85	113k	86	14	-	1	1	1
2	31k	14.5M	4.7M	46k	14	34	52	.92	.90	.91	166k	86	14	1	1	1	1
3	26k	11.9M	4.3M	97k	21	25	54	.85	.82	.84	180k	86	14	-	.93	.99	.96
4	18k	9.2M	3.2M	83k	23	29	48	.87	.85	.86	82k	88	12	-	1	1	1
5	14k	6M	1.9M	99k	20	38	43	.88	.86	.86	46k	90	10	-	1	1	1
6	15k	6M	2M	116k	23	38	39	.87	.87	.87	21k	73	24	2	1	1	1
7	6.7k	2.5M	748k	138k	29	39	32	.86	.86	.86	11k	69	31	-	1	1	1
8	6.8k	2.1M	657k	139k	21	33	46	.88	.87	.88	9.8k	63	37	-	1	1	1
9	6.3k	1.7M	744k	87k	13	17	70	.90	.85	.87	12k	84	16	-	.99	.99	1
10	3k	511k	250k	106k	14	38	48	.88	.86	.87	1.9k	45	55	-	1	1	1
11	798	109k	71k	23k	20	52	28	.84	.85	.84	-	-	-	-	-	-	-
12	579	25k	10k	8.9k	17	46	37	.87	.88	.87	270	45	55	-	1	1	1

Table 2: Left: Statistics on the number of unique DetectorIds, volume of alerts, and volume of incidents across 12 sampled regions over two-weeks. Middle: Triage model statistics, including the number of graded incidents (“Supp”), the distribution of grades (i.e., TP, FP, and BP), and model performance metrics evaluated through macro precision, recall, and F1 score. The triage statistics are sourced from numerous first and third party providers, with a large proportion of BP and FP incidents coming from third party providers. Right: Remediation model statistics, including the number of alerts with an action label (“Supp”), the distribution of contain account, isolate device, and stop virtual machine actions, and model performance metrics.

7.2 GUIDE Dataset

GUIDE includes over 13M pieces of evidence across 33 entity types, encompassing 1.6M alerts and 1M incidents over a two-week period. Each incident is annotated with a ground-truth triage label by customer security analysts, along with 26k alerts containing labels of remediation actions taken by customers. The dataset is derived from Region 2 (see Table 2) and includes telemetry across 6.1k organizations, featuring 9.1k unique custom and built-in DetectorIds across numerous security products, covering 441 MITRE ATT&CK techniques [44]. We divide the dataset into a train (70%) and test set (30%), both available as CSV files on Kaggle. The release of GUIDE provides an unparalleled opportunity for the development of GR systems and beyond, with CGR providing a foundational baseline. Additional details on GUIDE can be found in the Appendix.

7.3 Triage

The telemetry collection period for triage ranges from 7 to 180 days, varying by region due to computational constraints in incident pre-processing. Table 2 shows that the number of SOC-graded incidents (abbreviated as “Supp” for support) ranges from 8.9k to 139k with an imbalance among the three triage classes of 19% true positive, 35% false positive, and 46% benign positive (informational). While larger regions tend to have more graded incidents, several factors complicate the number of graded incidents reported during training: (1) larger regions with more organizations and detectors require more memory during OHE, reducing capacity for additional alerts; (2) the frequency and types of grading vary significantly across SOC and regions, and (3) we limit the number of graded examples per IncidentHash and triage label to 1k.

Offline results. In Table 2, we present the cross-region model triage performance at the point of maximum macro-F1 score on the precision-recall curve. The results show an average macro-F1 score

of 0.87, with a precision of 0.87, and a recall of 0.86. This performance demonstrates the model’s ability to effectively manage the complexities of the triage task, ranging from incidents involving a single alert to those comprising hundreds of alerts across numerous detectors and security products.

To evaluate CGR’s effectiveness at a more granular level, we examine performance across individual triage classifications in Region 2. The precision-recall curves in Figure 3 show that CGR consistently achieves high AUC scores across triage labels. The majority of misclassifications (81.1%) occur when incidents are categorized as BP instead of TP or FP, or vice versa—a discrepancy mainly due to the lack of a standardized definition for BP incidents, leading to classification inconsistencies across and within SOC. However, the more critical incident misclassification of TP as BP or FP is rare (2.4%), as shown in Table 3.

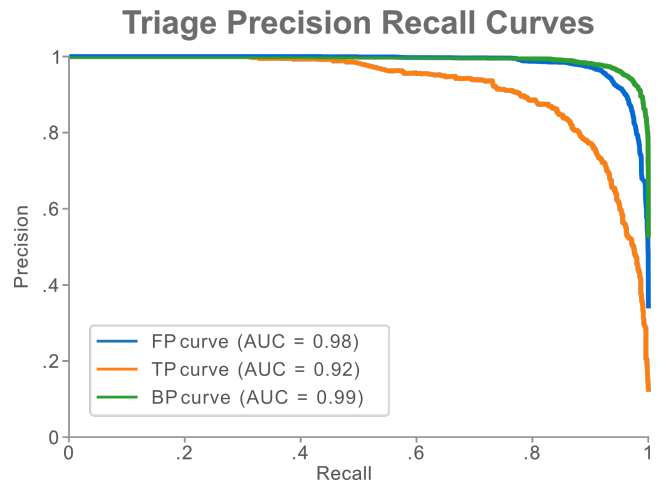


Figure 3: PR curves of triage performance in Region 2

Actual / Predicted	TP	BP	FP
TP	929	170	50
BP	62	4,748	45
FP	66	221	2,869

Table 3: Confusion matrix of triage performance in Region 2. Diagonal values reflect correct classifications, highlighting strong model performance across triage classes.

Online results. Triage recommendations are inherently dynamic, adapting as new alerts are added to incidents. However, as shown in Figure 2, since the vast majority of incidents are relatively short-lived and involve only a few alerts, only 2% of incidents undergo a change in their initial triage recommendation. Furthermore, model training pipelines across regions exhibit some bias, as not all types of incidents and alerts are graded by SOC analysts. Coupled with a heightened precision threshold to ensure that recommendations are correct 90% of the time, the triage models effectively cover 41% of incidents across regions.

7.4 Investigation

In collaboration with security experts at Microsoft, we manually evaluated our similar incident recommendations due to an absence of a definitive ground truth. For this evaluation, we randomly selected 1k incidents varying in size, detectors, products, organizations, and regions. Security researchers were then tasked with randomly selecting a similar incident recommendation for each reference incident, and judging its relevance based on criteria such as shared attack patterns, indicators of compromise, and entity types.

Offline results. The assessment found that 94% of the recommended incidents were relevant, with only 2% deemed dissimilar. The quality of recommendations tends to diminish for smaller organizations with limited historical data. Similarly, larger incidents characterized by hundreds of alerts across multiple products and detectors also show decreased recommendation quality due to their rarity. To address this, a cosine similarity threshold of 0.9 was identified as the cut-off threshold to ensure that only relevant recommendations are presented to customers. Across regions, we find that 98% of all incidents have one or more recommendations.

7.5 Remediation

We collect 180 days of telemetry, utilizing a preprocessing pipeline for alerts that is significantly simpler than the one used for incident-based triage. Unlike incidents, alerts do not require sampling due to their less computationally intensive preprocessing, allowing for a higher volume of training data, particularly in larger regions as shown in Table 2. The volume of actioned alerts varies widely across regions, from a few hundred to 180k, with a notable imbalance among the remediation classes—67% are contain account (CA), 23% isolate device (ID), and less than 1% stopping virtual machines (VM). Future work to support remediation actions, such as quarantining files, deleting emails, and blocking IPs/URLs is ongoing.

Offline results. We achieve an impressive average cross-region macro-F1 score of 0.99. This high score reflects the relative simplicity of predicting the appropriate remediation action for a single alert compared to the complexities of incident triaging in our dataset.

Online results. While we achieve notable offline results, the collected data does not fully capture the SOC experience. A majority of alerts are not actioned by analysts, and not all alerts fit the three predefined remediation action types. As a result, the model’s coverage averages 62% across regions. However, as we integrate new remediation actions, the system’s coverage will naturally increase.

8 Deployment

COPILOT GUIDED RESPONSE has been successfully deployed across the world, serving thousands of Microsoft Defender XDR customers since its launch in April 2024. CGR has generated millions of guided response recommendations for triage, investigation, and remediation tasks, receiving a ‘positive’ user response rate of 89%, based on the confirmation or dismissal of recommendations.

Our deployment infrastructure leverages a Synapse-based PySpark cluster, customized to each geographical region. This infrastructure includes: (a) an ADLS database ensuring both accessibility and secure management of telemetry; (b) an Azure Synapse backend that provides a robust framework for deployment; (c) an XXL PySpark pool featuring 60 executors, each equipped with 64 CPU cores and 400GB of RAM; (d) autoscaling to adjust executors based on fluctuating load; and (e) automated re-execution of failed jobs to ensure continuous coverage. Due to the absence of native support for model monitoring, versioning, and storage within Synapse, we developed a custom infrastructure to support these capabilities.

9 Conclusion

COPILOT GUIDED RESPONSE (CGR) represents the first time a cybersecurity company has openly discussed an industry-scale guided response framework. CGR significantly enhances SOC operations by guiding security analysts through crucial investigation, triaging, and remediation tasks, adeptly handling everything from simple alerts to complex incidents. The performance of CGR has been rigorously evaluated through internal testing, collaboration with Microsoft security experts, and extensive customer feedback, demonstrating its effectiveness across all three tasks. Deployed globally within Microsoft Defender XDR, CGR generates millions of guided response recommendations weekly, with 89% of user interactions receiving positive feedback. In addition, we release GUIDE, the largest publicly available collection of real-world security incidents, comprising 13 million pieces of evidence across one million incidents, each annotated with ground-truth triage labels by customer security analysts. As the first resource of its kind, GUIDE sets a new standard for advancing the development and evaluation of guided response systems and beyond.

Acknowledgments

We thank our colleagues who supported this research, including Shira Shacham, Yuval Derman, Oren Saban, Noa Bratman, Inbar Rotem, Omri Kantor, Itamar Karavani, Mari Mishel, Yuval Katav, Niv Zohar, Lior Camri, Leeron Luzzatto, Anna Karp, Ido Nitzan, Corina Feurstein, Ya’ara Cohen, Nadia Tkach Mendes, Gil Shmaya, Pawel Partyka, Shachaf Levy, Blake Strom, along with many others.

References

- [1] Khalid Alsubhi, Issam Aib, and Raouf Boutaba. 2012. FuzMet: A fuzzy-logic based alert prioritization engine for intrusion detection systems. *International Journal of Network Management* 22, 4 (2012), 263–284.
- [2] Khalid Alsubhi, Ehab Al-Shaer, and Raouf Boutaba. 2008. Alert prioritization in intrusion detection systems. In *NOMS 2008-2008 IEEE Network Operations and Management Symposium*. IEEE, 33–40.
- [3] Hilala Alturkistani and Mohammed A El-Affendi. 2022. Optimizing cybersecurity incident response decisions using deep reinforcement learning. *International Journal of Electrical and Computer Engineering* 12, 6 (2022), 6768.
- [4] Muhamad Erza Aminanto, Tao Ban, Ryoichi Isawa, Takeshi Takahashi, and Daisuke Inoue. 2020. Threat alert prioritization using isolation forest and stacked auto encoder with day-forward-chaining analysis. *IEEE Access* 8 (2020), 217977–217986.
- [5] Andy Applebaum, Shawn Johnson, Michael Limiero, and Michael Smith. 2018. Playbook oriented cyber response. In *2018 National Cyber Summit (NCS)*. IEEE, 8–15.
- [6] Tao Ban, Ndichu Samuel, Takeshi Takahashi, and Daisuke Inoue. 2021. Combat security alert fatigue with ai-assisted techniques. In *Proceedings of the 14th Cyber Security Experimentation and Test Workshop*. 9–16.
- [7] May Bashendy, Ashraf Tantawy, and Abdelkarim Erradi. 2023. Intrusion response systems for cyber-physical systems: A comprehensive survey. *Computers & Security* 124 (2023), 102984.
- [8] Joey Caparas, Samantha Robertson, Aditi Srivastava, Denise Vangel, Stephanie Savell, and Macky Cruz. 2024. What is Microsoft Copilot for Security?
- [9] Forrester Consulting. 2020. The 2020 State Of Security Operations.
- [10] Rahul Duggal, Scott Freitas, Sunny Dhamnani, Duen Horng Chau, and Jimeng Sun. 2021. Har: Hardness aware reweighting for imbalanced datasets. In *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 735–745.
- [11] Rahul Duggal, Scott Freitas, Cao Xiao, Duen Horng Chau, and Jimeng Sun. 2020. REST: Robust and Efficient Neural Networks for Sleep Monitoring in the Wild. In *Proceedings of The Web Conference 2020*. 1704–1714.
- [12] Charles Feng, Shuning Wu, and Ningwei Liu. 2017. A user-centric machine learning framework for cyber security operations center. In *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, 173–175.
- [13] Bingrui Foo, Y-S Wu, Y-C Mao, Saurabh Bagchi, and Eugene Spafford. 2005. ADEPTS: Adaptive intrusion response using attack graphs in an e-commerce environment. In *2005 International Conference on Dependable Systems and Networks (DSN'05)*. IEEE, 508–517.
- [14] Muriel Figueredo Franco, Bruno Rodrigues, Eder John Scheid, Arthur Jacobs, Christian Killer, Lisandro Zambenedetti Granville, and Burkhard Stiller. 2020. SecBot: a business-driven conversational agent for cybersecurity planning and management. In *2020 16th international conference on network and service management (CNSM)*. IEEE, 1–7.
- [15] Scott Freitas, Yuxiao Dong, Joshua Neil, and Duen Horng Chau. 2021. A Large-Scale Database for Graph Representation Learning. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [16] Scott Freitas, Rahul Duggal, and Duen Horng Chau. 2022. MalNet: A large-scale image database of malicious software. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 3948–3952.
- [17] Dianne Gali, Samantha Robertson, Daniel Simpson, and Stephanie Savell. 2024. Triage and investigate incidents with guided responses from Microsoft Copilot in Microsoft Defender.
- [18] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical provenance analysis for endpoint detection and response systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1172–1189.
- [19] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. Nodotze: Combatting threat alert fatigue with automated provenance triage. In *network and distributed systems security symposium*.
- [20] Shuang Huang, Chun-Jie Zhou, Shuang-Hua Yang, and Yuan-Qing Qin. 2015. Cyber-physical system security for networked industrial processes. *International Journal of Automation and Computing* 12, 6 (2015), 567–578.
- [21] IBM. 2020. IBM Automated Cyber Threat Triage and Response. <https://events.afcea.org/afceacyber20/CUSTOM/pdf/IBM%20Automated%20Cyber%20Threat%20Triage%20and%20Response%20Solution%20Datasheet.pdf>
- [22] Zakira Inayat, Abdullah Gani, Nor Badrul Anuar, Muhammad Khurram Khan, and Shahid Anwar. 2016. Intrusion response systems: Foundations, design, and challenges. *Journal of Network and Computer Applications* 62 (2016), 53–74.
- [23] Yuxuan Jiang, Chaoyun Zhang, Shilin He, Zhihao Yang, Minghua Ma, Si Qin, Yu Kang, Yingnong Dang, Saravan Rajmohan, Qingwei Lin, et al. 2024. Xpert: Empowering incident management with query recommendations via large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [24] Joseph Khoury and Mohamed Nassar. 2020. A hybrid game theory and reinforcement learning approach for cyber-physical systems security. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 1–9.
- [25] Irina Kraeva and Gulnara Yakhyayeva. 2021. Application of the metric learning for security incident playbook recommendation. In *2021 IEEE 22nd International Conference of Young Professionals in Electron Devices and Materials (EDM)*. IEEE, 475–479.
- [26] Ryuta Kremer, Prasanna N Wudali, Satoru Momiyama, Toshinori Araki, Jun Furukawa, Yuval Elovici, and Asaf Shabtai. 2023. IC-SECURE: Intelligent System for Assisting Security Experts in Generating Playbooks for Automated Incident Response. *arXiv preprint arXiv:2311.03825* (2023).
- [27] Xuan Li, Chunjie Zhou, Yu-Chu Tian, and Yuanqing Qin. 2018. A dynamic decision-making approach for intrusion response in industrial control systems. *IEEE Transactions on Industrial Informatics* 15, 5 (2018), 2544–2554.
- [28] Tao Lin. 2018. A Data Triage Retrieval System for Cyber Security Operations Center. (2018).
- [29] Yushan Liu, Xiaokui Shu, Yixin Sun, Jiyong Jang, and Prateek Mittal. 2022. RAPID: Real-Time Alert Investigation with Context-aware Prioritization for Efficient Threat Discovery. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 827–840.
- [30] Allie Mellen, Joseph Blankenship, Sarah Morana, and Michael Belden. 2024. The Forrester Wave™: Extended Detection And Response Platforms, Q2 2024.
- [31] Microsoft. 2024. Microsoft Copilot for Security.
- [32] Palo Alto Networks. 2024. Cortex Related Incidents. <https://docs-cortex.paloaltonetworks.com/r/Cortex-XSOAR/6.5/Cortex-XSOAR-Administrator-Guide/Manage-Related-Incidents>
- [33] Palo Alto Networks. 2024. Cortex XSOAR Incident Similarity. <https://xsoar.pan.dev/docs/reference/scripts/d-bot-find-similar-incidents>
- [34] Palo Alto Networks. 2024. Suggested Remediation. <https://docs-cortex.paloaltonetworks.com/r/Cortex-XDR/Cortex-XDR-Pro-Administrator-Guide/Remediate-Changes-from-Malicious-Activity>
- [35] Tien N Nguyen and Raymond Choo. 2021. Human-in-the-loop xai-enabled vulnerability detection, investigation, and mitigation. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1210–1212.
- [36] Jonathan Oliver, Raghav Batta, Adam Bates, Muhammad Adil Inam, Shelly Mehta, and Shugao Xia. 2024. Carbon Filter: Real-time Alert Triage Using Large Scale Clustering and Fast Search. *arXiv preprint arXiv:2405.04691* (2024).
- [37] Alina Oprea, Zhou Li, Robin Norris, and Kevin Bowers. 2018. Made: Security analytics for enterprise threat detection. In *Proceedings of the 34th Annual Computer Security Applications Conference*. 124–136.
- [38] Vihanga Heshan Perera, Amila Nuwan Senarathne, and Lakmal Rupasinghe. 2019. Intelligent soc chatbot for security operation center. In *2019 International Conference on Advancements in Computing (ICAC)*. IEEE, 340–345.
- [39] Yuanqing Qin, Qi Zhang, Chunjie Zhou, and Naixue Xiong. 2018. A risk-based dynamic decision-making approach for cybersecurity protection in industrial control systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 50, 10 (2018), 3863–3870.
- [40] Alireza Shameli-Sendi, Naser Ezzati-Jivan, Masoume Jabbarifar, and Michel Dagenais. 2012. Intrusion response systems: survey and taxonomy. *Int. J. Comput. Sci. Netw. Secur* 12, 1 (2012), 1–14.
- [41] Alireza Shameli-Sendi, Habib Louafi, Wenbo He, and Mohamed Cheriet. 2016. Dynamic optimal countermeasure selection for intrusion response system. *IEEE Transactions on Dependable and Secure Computing* 15, 5 (2016), 755–770.
- [42] Awalin Sopan, Matthew Berninger, Murali Mulakaluri, and Raj Katakam. 2018. Building a Machine Learning Model for the SOC, by the Input from the SOC, and Analyzing it for the SOC. In *2018 IEEE Symposium on Visualization for Cyber Security (VizSec)*. IEEE, 1–8.
- [43] Zheni S Stefanova and Kandethody M Ramachandran. 2018. Off-policy q-learning technique for intrusion response in network security. *World Academy of Science, Engineering and Technology, International Science Index* 136 (2018), 262–268.
- [44] Blake E Strom, Andy Applebaum, Doug P Miller, Kathryn C Nickels, Adam G Pennington, and Cody B Thomas. 2018. Mitre att&ck: Design and philosophy. In *Technical report*. The MITRE Corporation.
- [45] Thomas Toth and Christopher Kruegel. 2002. Evaluating the impact of automated intrusion response mechanisms. In *18th Annual Computer Security Applications Conference, 2002. Proceedings*. IEEE, 301–310.
- [46] Chen Zhong, Tao Lin, Peng Liu, John Yen, and Kai Chen. 2018. A cyber security data triage operation retrieval system. *Computers & Security* 76 (2018), 12–31.
- [47] Saman A Zonouz, Himanshu Khurana, William H Sanders, and Timothy M Yardley. 2013. RRE: A game-theoretic intrusion response and recovery engine. *IEEE Transactions on Parallel and Distributed Systems* 25, 2 (2013), 395–406.

Appendix

A Dataset Overview

GUIDE represents the largest publicly available collection of real-world cybersecurity incidents, containing over 13 million pieces of evidence across 1.6 million alerts and 1M annotated incidents. Under the permissive CDLA-2.0 license, this dataset offers a unique opportunity for researchers and practitioners to develop and benchmark advanced ML models on authentic and comprehensive security telemetry. See Table 4 for a description of each dataset field.

We provide three hierarchies of data: (1) evidence, (2) alert, and (3) incident. At the bottom level, evidence supports an alert. For example, an alert may be associated with multiple pieces of evidence such as an IP address, email, and user details, each containing specific supporting metadata. Above that, we have alerts that consolidate multiple pieces of evidence to signify a potential security incident. These alerts provide a broader context by aggregating related evidences to present a more comprehensive picture of the potential threat. At the highest level, incidents encompass one or more alerts, representing a cohesive narrative of a security breach or threat scenario.

B Benchmarking

With the release of GUIDE, we aim to establish a standardized benchmark for guided response systems using real-world data. The primary objective of the dataset is to accurately predict incident triage grades—true positive (TP), benign positive (BP), and false positive (FP)—based on historical customer responses. To support this, we provide a training dataset containing 45 features, labels, and unique identifiers across 1M triage-annotated incidents. We divide the dataset into a train set containing 70% of the data and a test set with 30%, stratified based on triage grade ground-truth, OrgId, and DetectorId. We ensure that incidents are stratified together within the train and test sets to ensure the relevance of evidence and alert rows. The CSV files are hosted on Kaggle: <https://www.kaggle.com/datasets/Microsoft/microsoft-security-incident-prediction>

A secondary objective of GUIDE is to benchmark the remediation capabilities of guided response systems. To this end, we release 26k ground-truth labels for predicting remediation actions for alerts, available at both granular and aggregate levels. The recommended metric for evaluating research using the GUIDE dataset is macro-F1 score, along with details on precision and recall.

C Privacy

To ensure privacy, we implement a stringent anonymization process. Initially, sensitive values are pseudo-anonymized using SHA1 hashing techniques. This step ensures that unique identifiers are obfuscated while maintaining their uniqueness for consistency across the dataset. Following this, we replace these hashed values with randomly generated IDs to further enhance anonymity and prevent any potential re-identification. Additionally, we introduce noise to the timestamps, ensuring that the temporal aspects of the data cannot be traced back to specific events. This multi-layered approach, combining pseudo-anonymization and randomization, safeguards the privacy of all entities involved while maintaining the integrity and utility of the dataset for research and development purposes.

Feature	Description
Id	Unique ID for each OrgId-IncidentId pair
OrgId	Organization identifier
IncidentId	Organizationally unique incident identifier
AlertId	Unique identifier for an alert
Timestamp	Time the alert was created
DetectorId	Unique ID for the alert generating detector
AlertTitle	Title of the alert
Category	Category of the alert
MitreTechniques	MITRE ATT&CK techniques involved in alert
IncidentGrade	SOC grade assigned to the incident
ActionGrouped	SOC alert remediation action (high level)
ActionGranular	SOC alert remediation action (fine-grain)
EntityType	Type of entity involved in the alert
EvidenceRole	Role of the evidence in the investigation
Roles	Additional metadata on evidence role in alert
DeviceId	Unique identifier for the device
DeviceName	Name of the device
Sha256	SHA-256 hash of the file
IpAddress	IP address involved
Url	URL involved
AccountSid	On-premises account identifier
AccountUpn	Email account identifier
AccountObjectId	Entra ID account identifier
AccountName	Name of the on-premises account
NetworkMessageId	Org-level identifier for email message
EmailClusterId	Unique identifier for the email cluster
RegistryKey	Registry key involved
RegistryValueName	Name of the registry value
RegistryValueData	Data of the registry value
ApplicationId	Unique identifier for the application
ApplicationName	Name of the application
OAuthApplicationId	OAuth application identifier
ThreatFamily	Malware family associated with a file
FileName	Name of the file
FolderPath	Path of the file folder
ResourceIdName	Name of the Azure resource
ResourceType	Type of Azure resource
OSFamily	Family of the operating system
OSVersion	Version of the operating system
AntispamDirection	Direction of the antispam filter
SuspicionLevel	Level of suspicion
LastVerdict	Final verdict of threat analysis
CountryCode	Country code evidence appears in
State	State of evidence appears in
City	City evidence appears in

Table 4: Description of each column in the GUIDE dataset